

Design and Analysis of Algorithms

Q1) Consider the following recursive algorithm

```
K(n){
    IF (n = 1)
        RETURN 1
    ELSE
        SUM = 0
        FOR( i = 1 to n - 1 )
            SUM = SUM + (K(i) + K(n - i))/3 + n/2
        RETURN SUM
}
```

Convert the recursive code given above into bottom up iterative dynamic programming algorithm.

Solution:

Let's dry run the above code for better understanding.

$n=1, K(1) = 1$

$n=2, K(2) = (K(1) + K(1))/3 + 2/2 = 2/3 + 1 = 1.66$

$n=3, K(3) = [(K(1) + K(2))/3 + 2/2] + [(K(2) + K(1))/3 + 3/2] = [(1 + 1.66)/3 + 1] + [(1.66 + 1)/3 + 1.5] = [1.88] + [2.38] = 3.26$

We can see that we need value of $K(2)$ multiple times. If we can use memoization to store $K[2]$ in an array then we can save repeated computations. The above recursive code is calculating $K(2)$ again and again.

We can create an array $DP[i]$ which saves solution for i . We will need two for loops, one for calculating smaller subproblems and other for calculating answer of one subproblem.

```
Iterative(n){
    DP[1] = 1 // smallest subproblem
    FOR( j = 2 to n ) // j represents size of subproblems
        SUM = 0
        FOR( i = 1 to j - 1 )
            SUM = SUM + (DP[i] + DP[j - i])/3 + j/2
        DP[j] = SUM
    RETURN DP[n]
}
```

Q2) Consider the following recursive algorithm

```

P(n, k){
    IF (k > n)
        RETURN 0
    IF (k == n || k == 0)
        RETURN 1

    RETURN P(n-1, k-1) + P(n-1,k)
}

```

Convert the recursive code given above into bottom up iterative dynamic programming algorithm.

Solution

```

P(n, k){
    C[0] = 1

    FOR( i = 1 to n )
        FOR( j = 0 to min(i, k) )
            IF(j == 0 || j == i)
                C[i][j] = 1
            ELSE
                C[i][j] = C[i-1][j] + C[i-1][j-1]

    RETURN C[n][k]
}

```

Q3) Given a matrix $M * N$ of integers where each cell has a cost associated with it, the cost can also be negative. Find the minimum cost to reach the last cell ($M-1, N-1$) of the matrix from its first cell (0,0). We can only move one unit right (Column No + 1), one unit down (Row No + 1), and one unit in bottom diagonal (Row No + 1, Column No + 1). For example from index (i , j) you can move to (i , j+1), (i+1 , j), and (i+1, j+1) where i = row no and j = column no.

Example

4	7	8	6	4
-6	7	3	9	2
3	8	1	-2	4
7	1	7	3	7
2	9	8	9	3

4	7	8	6	4
-6	7	3	9	2
3	8	1	-2	4
7	1	7	3	7
2	9	8	9	3

Path with minimum cost = 4 -> -6 -> 7 -> 1 -> -2 -> 3 -> 3 = 10

Provide a Dynamic Programming solution for it.

a) Provide recurrence for sub-problem

a) Provide pseudo code for DP solution

Solution

$$C(i, j) = \text{Min} (C[i][j-1], C[i-1][j], C[i-1][j-1]) + A[i][j]$$

a) Provide pseudo code for DP solution

// Initialization

for(i: 1 to m)

$$C[i][0] = C[i-1][0] + C[i][0]$$

for(i: 1 to n)

$$C[0][i] = C[0][i-1] + C[0][i]$$

for(i: 1 to m)

for (j: 1 to n)

$$C[i][j] = \text{Min} (C[i][j-1], C[i-1][j], C[i-1][j-1]) + A[i][j]$$

Problem 4

You are going to Europe by road! Wow. And its one single road (wow!). You start on the road at distance $d_1 = 0$. There are many stopovers on this roads and these are located at distances $d_1 < d_2 < \dots < d_n$. You are driving the car yourself and will need to stop at some of these stopovers. In the end, you will definitely stop at the final stopover at distance d_n .

Your aim is to travel 500 kms every day (you make stopover at night), as this seems to be the ideal distance that balances the fatigue with the pleasure of travelling the most. However, it's not always possible to do so because the stopovers are not always 500 kms apart from each other. (If they were, you could travel 500 Kms and make a stop, and so on). Being of a quantitative bent of mind, you have formulated that if you travel α miles in a day, the amount of 'loss' (in terms of more fatigue and less joy) for that day is $(500 - \alpha)^2$.

You want to make stops such that this total loss is minimized — that is, the sum, over all travel days, of the daily losses is minimized. Give an efficient DP algorithm that determines the optimal sequence of stopovers at which to stop so as to minimize loss.

- (a) Define a subproblem
- (b) Write a recurrence
- (c) Write a bottom up algorithm to compute the subproblem solutions
- (d) Write a method to retrieve the optimal stopover sequence that produces the optimal (minimum) value of loss.

Solution

a. Sub-Problem: $dp[i]$ is the minimum fatigue the person can have if he stops at the i th stop. b.

b. Recurrence: $dp[i] = \min(\text{sq}(500 - (X[i] - X[j])) + dp[j])$ where $0 \leq j < i$

Problem 5

Now at this point you have become an algorithm expert and it's the right time to increase the difficulty level. You have given an array $M = \{1, 2, 3, 8, 6, 7, 9, 10, 2, 1\}$. You have to calculate the longest increasing with decreasing subsequence in the array. Let's understand what a subsequence is through an example.

1, 3, 6, 10, 1 à Subsequence

1, 2, 3, 5 à Sequence

There is a subtle difference between subsequence and sequence. In Subsequence we can skip some elements but it should be done in increasing order, like 1 3 2 5 6 is not subsequence. Order of elements in parent array should remain same in subsequence. Like here 3 comes after 1 so there exist no subsequence in which 3 comes before 1.

In the problem you have to find such subsequence which has 2 parts, whose first part is **increasing** in order and second part is **decreasing** in order and it is the **longest** subsequence.

Here, (red is increasing subsequence and green is decreasing subsequence)

- i) 1 2 3 8 9 10 2 1
- ii) 1 2 3 6 7 9 10 2 1

are one of such subsequences but we can see that second subsequence is longest so we will choose this one

- 1) Define a subproblem for it.
- 2) Provide its recurrence.
- 3) Pseudo code for the algorithm devised.
- 4) Run time complexity

Solution

We will be having LIS and LDS , Longest increasing Sequence at *ending* index i and Longest decreasing Subsequence *Starting* at index i

= MAX(LIS[i] , DIS[i]) where i will range from 0 to size of array.

Problem 6

You are fascinated with stars and your friend challenges you with the algorithms question. In Problem you are given a m * m matrix containing only 1 and 0. Your challenge is to find biggest star in matrix in min amount of time and computations. For example in the following matrix the biggest start is highlighted, it is M[5][4] with length = 3. The star at M[5][4] entry with length 3 means all of the following entries have 1.

M[5][3] , M[5][2], M[5][1], //same row, previous columns

M[5][5], M[5][6], M[5][7], //same row, next columns

M[4][4], M[3][4], M[2][4], //same column, previous rows

M[6][4], M[7][4], M[8][4], //same column, next rows

M[4][3], M[3][2], M[2][1], //diagonal

M[6][5], M[7][6], M[8][7], // diagonal

M[6][3], M[7][2], M[8][1] // diagonal

M[4][5], M[3][6], M[2][7], // diagonal

1	1	0	1	1	0	1	1	0	1
1	1	1	0	1	1	1	0	1	0
0	1	0	1	1	1	0	0	0	1
0	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	0	0
1	0	1	0	1	1	1	0	0	1
0	1	0	0	1	0	1	1	1	1

0	1	1	1	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---

- A) DP Recurrence
- B) Time Complexity for DP Solution
- C) Time Complexity for Naive Solution

Solution

Given a 2D matrix **M** of binary digits, we have to find the size of the largest Star of 1s in it. Its DP solution is quite simple

As we simply populate 8 , 2D arrays up $\square\square$ down $\square\square$ right $\square\square$ left $\square\square$ and 4 diagonals in a single pass.

Recurrence: $Up[i][j] = Up[i-1][j] + 1$ if $Matrix[i][j] == 1$, same for other matrices
 Result = $MAX(MIN(up[i][j] , down[i][j] , right[i][j] , left[i][j] , Dig1[i][j] , Dig2[i][j] , Dig3[i][j] , Dig4[i][j]))$

Greedy Algorithms

Q7) Consider the problem of making change from n cents using the fewest coins when the available coins are quarters (25 cents), dimes (10 cents), nickels (5 cents) and pennies (1 cent). Does the greedy strategy of outputting the largest coin that does not exceed the amount of change that must still be returned yield an optimal solution?

Solution:

Yes, this strategy does yield an optimal solution. It is easily seen that this greedy algorithm can be implemented in $O(n)$ time.

Greedy Choice Property: We show that there exists an optimal solution for making n cents that begins with the largest coin whose value is $\leq n$ cents. Let S be the set of coins given as change by some optimal solution to the problem of making change for n cents. Suppose that the largest denomination among the coins in S is $k \in \{Q, D, N, P\}$. Let i be the first coin chosen by the greedy algorithm. If $k = i$ then the solution begins with a greedy choice. If $k \neq i$ then we must show that there is another optimal solution S' that begins with coin i. Observe that since the greedy algorithm picks the largest denomination that can be used, it follows that $k < i$. Thus $i \neq P$. We now consider each of the remaining cases.

Case 1: $i = N$. Thus the solution S must have value of 5 cents or more. Since the largest coin in S must be less than a nickel, S uses only pennies, and hence must have at least 5 pennies. Create S' from S by replacing 5 pennies by a nickel. $|S'| < |S|$ contradicting the optimality of S.

Case 2: $i = D$. Thus the solution S must have value of 10 cents or more. Since the largest coin in S must be less than a dime, S uses only pennies and nickels. The only way to create 10 cents or more with pennies and nickels must use either 2 nickels, 1 nickel and 5 pennies, or 10 pennies. In all three cases we can create S' from S by replacing this subset of coins that equals 10 cents by a dime. Again, $|S'| < |S|$ contradicting the optimality of S .

Case 3: $i = Q$. First suppose, that there is some combination of coins in S that sum to 25 cents. Then let $S' = S - \{\text{coins that sum to 25}\} \cup \{Q\}$ and observe $|S'| < |S|$. Next we consider the case when no subset of coins in S sums to 25. This can only occur if there are at least 3 dimes in S since that is required to obtain 30 cents or more without using a nickel or at least 5 pennies (in which case some subset of coins would add to 25 cents). So in this case let $S' = S - \{D + D + D\} \cup \{Q + N\}$. Again $|S'| < |S|$ contradicting the optimality of S . Hence in all three cases we proved not only that there exist some optimal solution that picks the same first coin as the greedy algorithm, but in fact, every optimal solution must pick this largest coin.

Q8) You are given a sequence of n songs where the i th song is l_i minutes long. You want to place all of the songs on an ordered series of CDs. (e.g. CD 1, CD 2, CD 3,, CD k) where each CD can hold m minutes. Furthermore,

1. The songs must be recorded in the given order, song 1, song 2, ... song n
2. All songs must be included
3. No song maybe split across CDs.

Your goal is to determine how to place them on the CDs as to minimize the number of CDs needed. Give the most efficient algorithm you can to find an optimal solution for this problem, analyze the time complexity.

Solution:

Here is the algorithm. Put as many songs (from song 1 to song g) on the first CD as possible without exceeding the m minute limit. Then recursively repeat this procedure for the remaining CDs. Since this just requires keeping a running sum in which each of the n song lengths are included once, clearly the above is an $O(n)$ algorithm. We now prove it is correct.

Greedy Choice Property:

Let S be an optimal solution in which the first CD holds songs 1 to k . The greedy algorithm puts songs 1 to g on the first CD. We now prove there is an optimal solution which puts the first g songs on the first CD. If $k = g$ then S is such a solution. Now suppose that $k \neq g$. Since the greedy algorithm puts as many songs on the first CD as possible, it follows that $k < g$. We construct a solution S' by modifying S to move all songs from $k + 1$ to g onto the first CD. We now argue that S' is a legal solution which is optimal. First note that the number of CDs used by S' is at most the number of CDs used by S . All that remains is to argue that S' is legal (i.e. no CD holds more than m minutes of songs). By definition of the greedy choice the first CD can hold songs 1 to g . Since S is a legal solution all of the CDs in it hold at most m minutes. For all but CD 1 the number of songs is only reduced and hence must still hold at most m minutes. Hence S' is legal.

Optimal Substructure Property:

Let P be the original problem with an optimal solution S . Then after putting songs 1 to g on CD 1, let P' be the subproblem of songs $(g + 1)$ to n . Let S' be an optimal solution to P' . Since,

$\text{cost}(S) = \text{cost}(S') + 1$, clearly an optimal solution to P includes within it an optimal solution to P' .

Q9) The input consists of n skiers with heights $p_1, \dots, p_n$, and n skis with heights $s_1, \dots, s_n$. The problem is to assign each skier a ski to minimize the AVERAGE DIFFERENCE between the height of a skier and his/her assigned ski. That is, if the skier i is given the ski a_i , then you want to minimize:

$$\sum_{i=1}^n \frac{|p_i - s_{a_i}|}{n}$$

(a) Consider the following greedy algorithm. Find the skier and ski whose height difference is minimized. Assign this skier this ski. Repeat the process until every skier has a ski. Prove or disprove that this algorithm is correct.

(b) Consider another greedy algorithm. Give the shortest skier the shortest ski, give the second shortest skier the second shortest ski, give the third shortest skier the third shortest ski, etc. Prove or disprove that this algorithm is correct.

HINT: One of the above greedy algorithms is correct and one is incorrect for the other.

Solution:

(a) This algorithm is INCORRECT for the problem of minimizing the average difference between the heights of skiers and their skis. Consider an instance with the following values: $p_1 = 5$; $p_2 = 10$; and $s_1 = 9$; $s_2 = 14$. The algorithm would pair p_1 with s_2 and p_2 with s_1 for a total cost of $1/2(1 + 9) = 5$. Pairing p_1 with s_1 and p_2 with s_2 yields a total cost of $1/2(4 + 4) = 4$.

(b) This algorithm is CORRECT. The proof is by contradiction. Assume the people and skis are numbered in increasing order by height. If the greedy algorithm is not optimal, then there is some input $p_1, \dots, p_n$; $s_1, \dots, s_n$ for which it does not produce an optimal solution. Let the optimal solution be $T = \{(p_1, s_{j(1)}), \dots, (p_n, s_{j(n)})\}$, and let the output of the greedy algorithm be $G = \{(p_1, s_1), \dots, (p_n, s_n)\}$. Beginning with p_1 , compare T and G . Let p_i be the first person who is assigned different skis in G than in T . Let s_j be the pair of skis assigned to p_i in T . Create solution T' by switching the ski assignments of p_i and p_j . By the definition of the greedy algorithm, $s_i \leq s_j$. The total cost of T' is given by

$$\text{Cost}(T') = \text{Cost}(T) - 1/n (|p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j|)$$

There are six cases to be considered. For each case, one needs to show that

$$(|p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j|) \geq 0$$

Case 1: $p_i \leq p_j \leq s_i \leq s_j$

$$(|p_i - s_j| + |p_j - s_i| - |p_i - s_i| - |p_j - s_j|) = (s_j - p_i) + (s_i - p_j) - (s_i - p_i) - (s_j - p_j) = 0$$

Case 2: $p_i \leq s_i \leq p_j \leq s_j$

Case 3: $pi \leq si \leq sj \leq pj$

Case 4: $si \leq sj \leq pj \leq pi$

Case 5: $si \leq pi \leq sj \leq pj$

Case 6: $si \leq pi \leq pj \leq sj$